Computação e Programação

Prof. MSc. Marcos de Souza

Roteiro

- Contextualização sobre os sistemas operacionais
- Linguagens de programação

Sistemas Operacionais

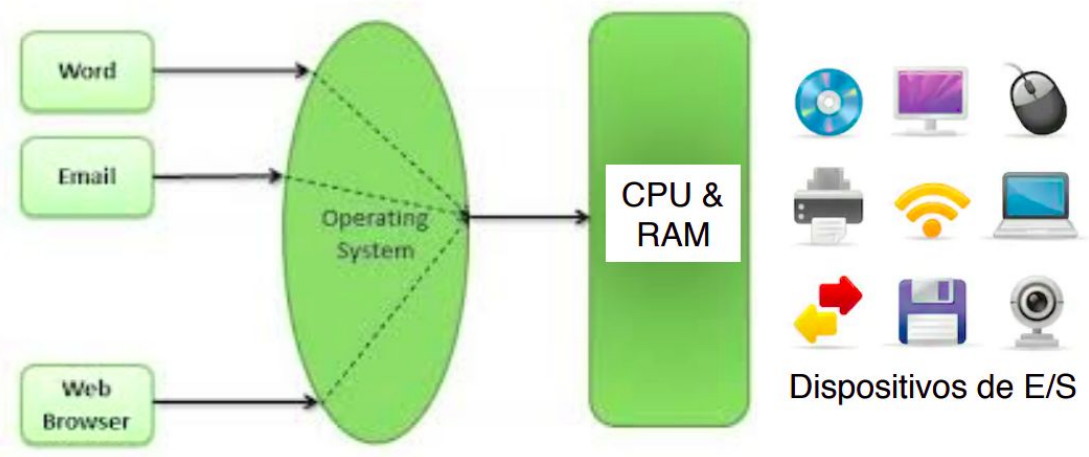
Características

- Trata-se de um software (segue todo o ciclo de desenvolvimento: engenharia de requisitos, modelagem e programação <- geralmente são construídos em linguagem C);
- Responsável por intermediar o usuário e o hardware (CPU, memória, HD, teclado, etc);
  - Através de drivers o SO conseguirá controlar de forma “eficiente” os hardwares
- Também é responsável pela execução de outros softwares;
- Possui uma interface de segurança (máquina virtual) que assegura o correto controle do hardware (ex.: evitar que um hardware exploda por superaquecimento)

Camadas de um SO (Tradicional)

Algumas Características

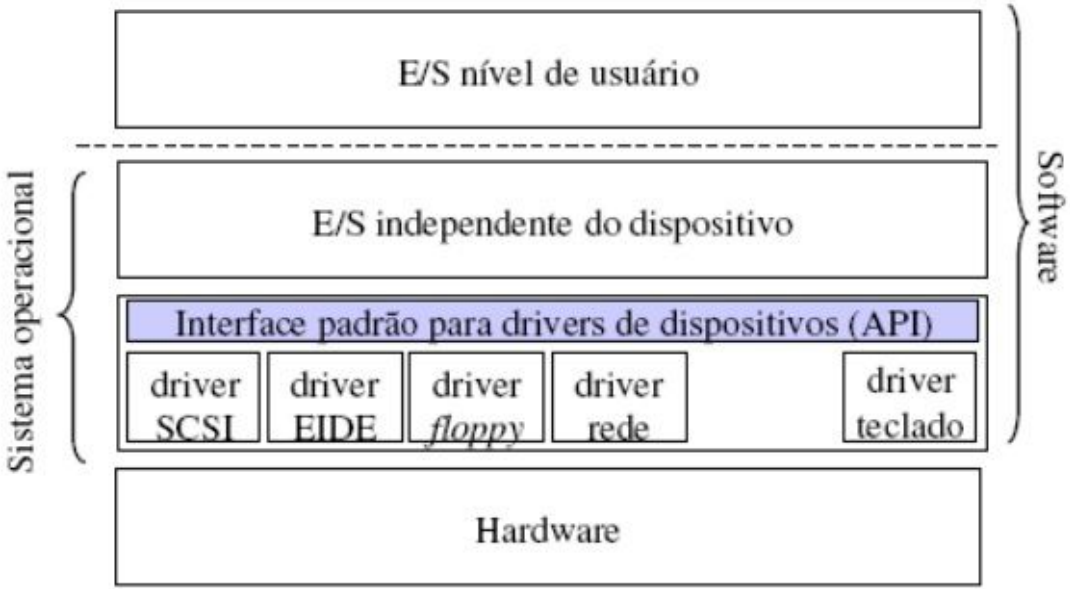
- Naturalmente os usuários acabam necessitando executar as atividades de forma paralela (concorrente), assim os SO tem de ser capazes de controlar o múltiplo acesso ao hardware;



Algumas Características

- Cada programa em execução intercala períodos de processamento (uso efetivo da CPU) com períodos de E/S.
- Seria um desperdício fazer a CPU esperar pelo término de cada E/S com um dispositivo
- Para tal, é importante que o SO possa inciar uma E/S e prosseguir com a execução de outra tarefa.
- Ou seja, multiprogramação aumenta a utilização da CPU.

Arquitetura do SO quanto a E/S

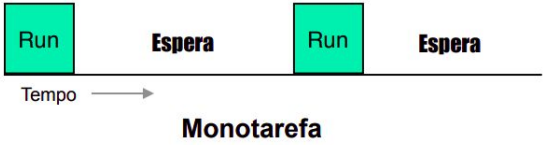


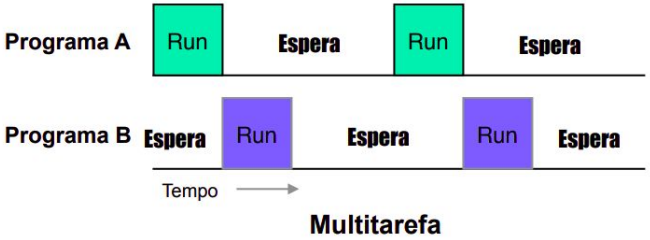
Virtualização de Sistemas

Uma outra abordagem é virtualizar o SO para que um seja responsável pelo controle do hardware e outro pela camada de aplicações

Categorias de Sistemas Operacionais

- Monoprogramação (monotarefa)
  - Cada processo só pode ser iniciado após o término do último processo
  - MS-DOS
- Multiprogramação (multitarefa)
  - É possível executar múltiplos processos
  - Os sistemas operacionais atuais





Mono-Usuário X Multi-Usuário

- Sistema Monousuário: Permite o login de apenas um usuário
  - não permite que mais de um usuário esteja “logado” simultaneamente
  - Ex: Windows 98, iOS, Android, Arduino, etc.
- Sistema Multiusuário: Admite e gerencia vários usuários
  - permite que mais de um usuário esteja “logado” no sistema simultaneamente.
  - Ex: Linux, Windows 2000, VMS
  - Sistema multi-usuário requer mecanismos de segurança (controle de acesso aos arquivos, e proteção para áreas de memória das tarefas em execução)

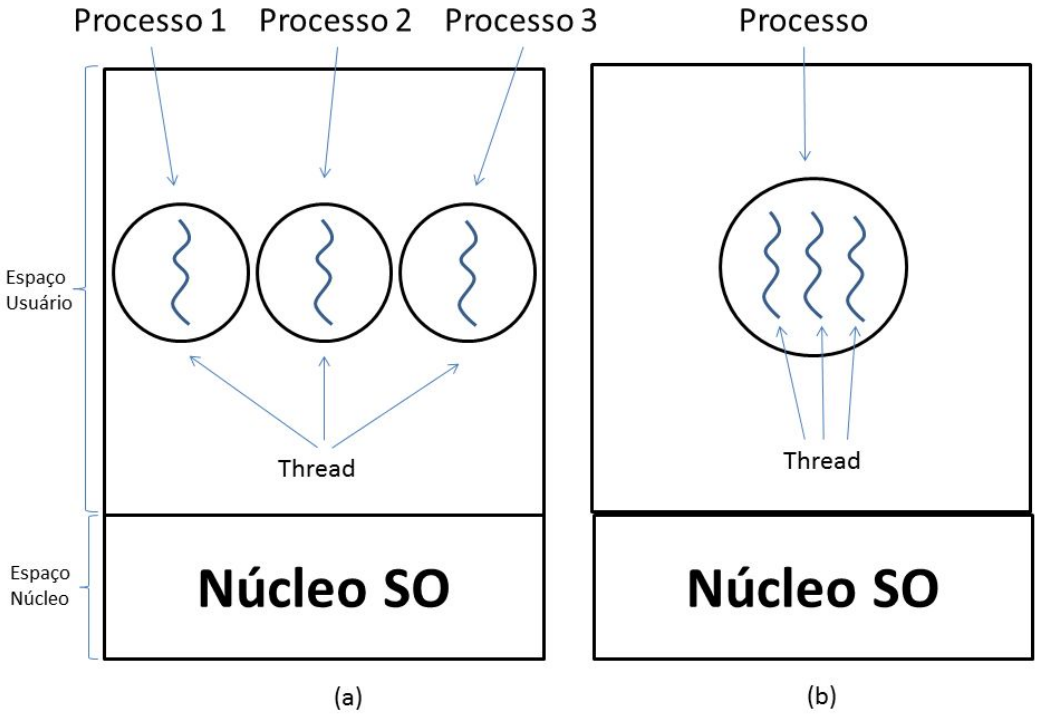
Mono-Usuário X Multi-Usuário

- Sistema Monoprocessado
  - Somente utiliza uma CPU
  - Pode ser multi-tarefa ou mono-tarefa
  - Ex: Windows 98
- Sistema Multiprocessado
  - Reconhece mais de uma CPU (multi-core, arquiteturas paralelas ou em rede)
  - Permite execução realmente paralela de tarefas
  - Ex: Windows 2000/NT/XP, Linux, MACH

Sistemas Embarcados

Sistemas embutidos em aparelhos dispositivos como micro-ondas, som, consoles de games, carros, etc. Possui apenas as funcionalidades para gerenciar os recursos do equipamento.

Multitarefa no SO



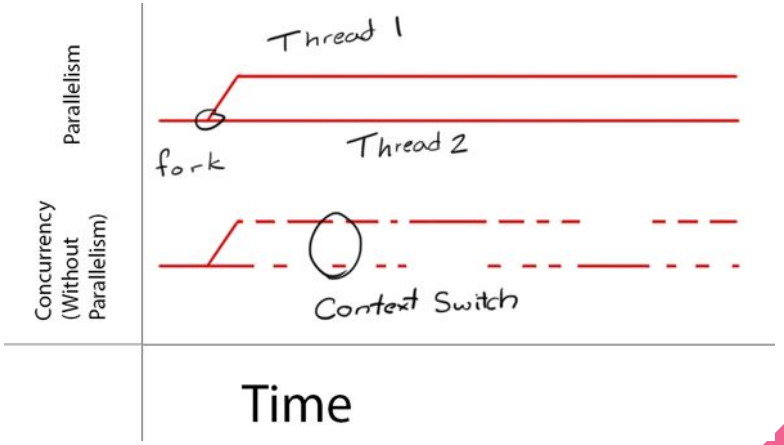
Threads

- Thread é um fluxo de execução dentro do processo.
- Multithread é uma forma de um processo dividir a si mesmo em duas ou mais tarefas que podem ser executadas **concorrentemente**.
- Existe dentro de um processo e usa os recursos do processo
- Tem o seu próprio fluxo de controle independente enquanto existir o processo pai e o SO dá suporte a ele
- Pode compartilhar os recursos do processo com outras threads igualmente independentes
- Um exemplo de uso de múltiplas threads é referente à leitura de um arquivo

Concorrência e Paralelismo

É preciso tomar cuidado com os termos que não significam a mesma coisa.

Enquanto o paralelismo se refere a uma execução independente a concorrência compartilha os mesmos recursos (ex.: acesso a um mesmo arquivo por diferentes processos)



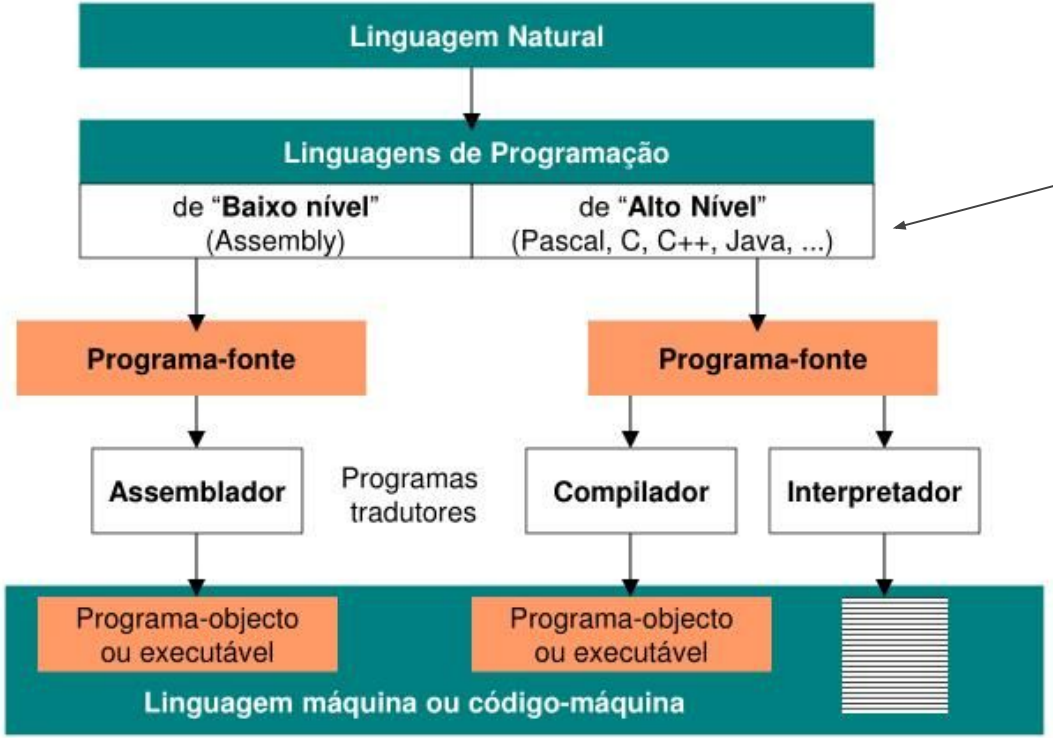
Linguagens de Programação

Linguagem de Programação

- Como desenvolvedores conhecemos os requisitos do sistema:
  - exemplo: validar se um determinado número possui valor entre 1 e 5
- A partir dessa linguagem ‘natural’ é possível escrever algoritmos (formalismos) que realizem essa operação em uma máquina:

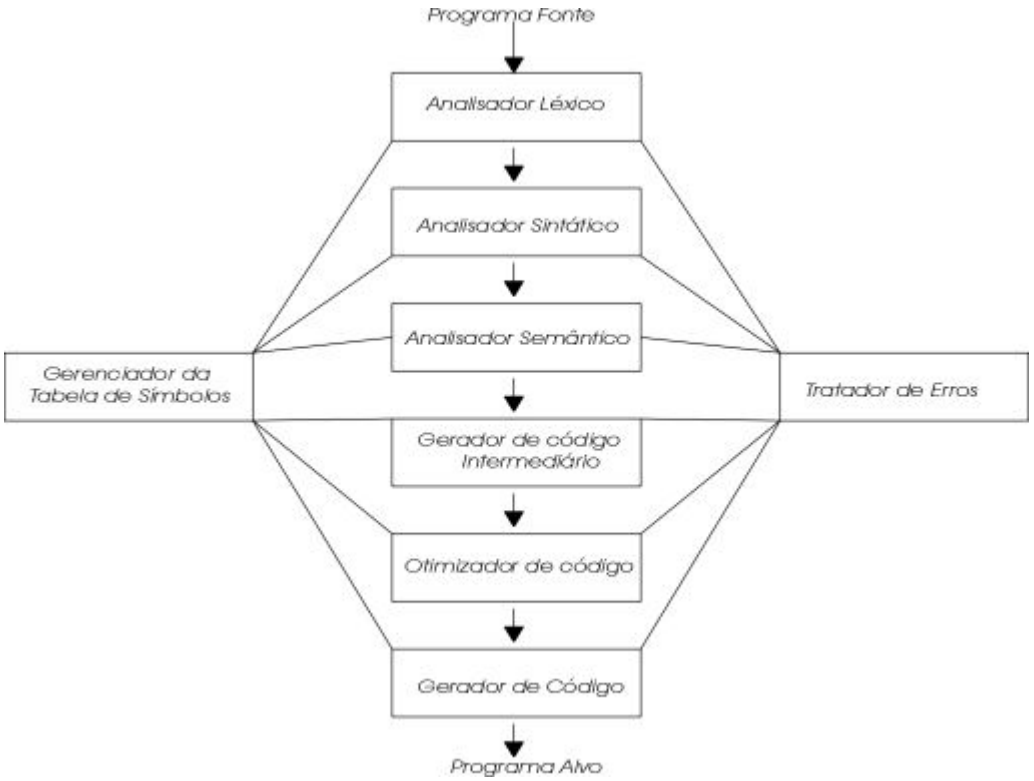
```
INÍCIO ALG VALIDA_NUMERO (NUMERO)

- SE NUMERO >= 1 E NUMERO <= 5
    - INFORME: “NÚMERO VÁLIDO”
- SENÃO
    - INFORME “NÚMERO INVÁLIDO”

FIM ALG VALIDA_NUMERO
```

O processo de tradução

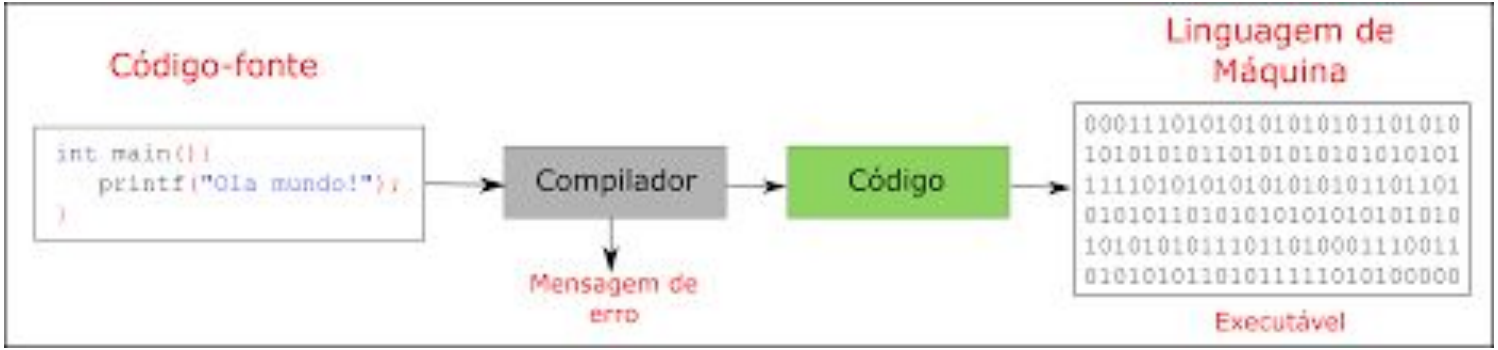


AQUI FICAM NOSSOS ALGORITMOS

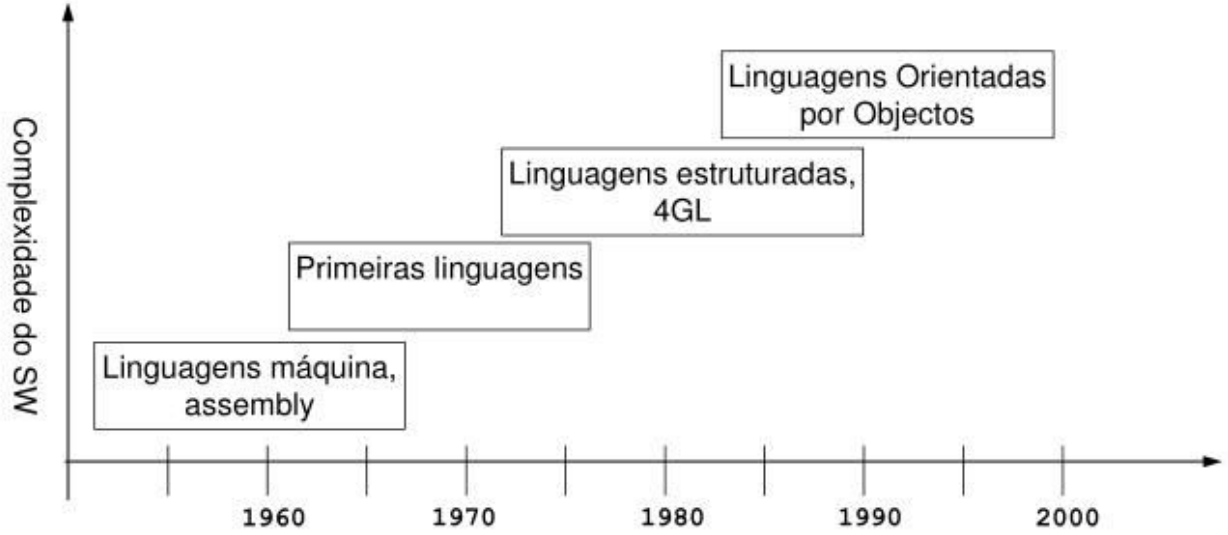
Fases de um Compilador



Tradução para o Código de Máquina



Histórico das Linguagens



Características

- **Ling. 1ª geração**
  - Dependentes da máquina, ao mais baixo nível de abstracção. •
  - Codificação em Assembly •
- **Ling. 2ª geração**
  - Oferecem algum nível de abstracção e Introduzem as Bibliotecas de Software
- **Ling. 3ª geração (Linguagens Estruturadas)**
  - Ricas em capacidades procedimentais e estruturas de dados.
  - Dividem-se em: • General Purpose High Order Languages (Pascal, C, ...) •
  - Object Oriented Languages (Java, Python, C++, C# ...)
- **Ling. 4ª geração**
  - Sintaxe distinta para controlo e representação de estruturas de dados
  - O seu elevado nível de abtração elimina a necessidade de especificação algoritmica exaustivamente detalhada
  - Combinam características procedimentais com não-procedimentais
  - Query Languages e Geradores de Aplicações (ex.: SQL)
- **Ling. 5ª geração**
  - Linguagens não-procedimentais que permitem a declaração do problema a resolver possuindo a própria linguagem mecanismos para a sua resolução
  - Linguagens declarativas (Prolog)

Paradigmas

Define um modelo ou norma a seguir pelas linguagens de programação baseadas no paradigma

- **Programação procedimental, sequencial, imperativa ou estrutural**

O programa é uma sequência de comandos ou instruções que o programador fornece ao computador e que este executa sequencialmente

- **Programação declarativa**

O programador declara o conhecimento que o computador deve ter necessário à resolução do problema e como este deve proceder para atingir a solução

- **Programação funcional**

Os programas baseiam-se em princípios matemáticos (funções e conjuntos)

- **Programação OO**

Surge como uma evolução da programação estruturada e modular. Os programas são substituídos por objetos que encapsulam dados e instruções